

LECTURE 20

Deadlock Avoidance and the Banker's Algorithm

Never leave a safe state

CSC323 · Operating Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- Define a safe state and a safe sequence — avoidance is "never leave a safe state"
- Walk through the RAG algorithm for avoidance in single-instance systems
- Learn the Banker's Algorithm's data structures: Available, Max, Allocation, Need
- Work a complete numeric example: run the Safety Algorithm by hand
- Work a complete Resource-Request Algorithm example, including a denied request

DEFINITION

Safe States and Safe Sequences

A system is in a safe state if there exists at least one order in which every currently existing process can run to completion — even if every process immediately asks for its declared maximum — without ever causing deadlock. That order is a safe sequence.

A sequence $P_1, P_2, \dots, P_n$ is safe if, for every P_i in that order, the resources P_i might still need can be satisfied using what's currently available plus everything held by processes earlier in the sequence.

NOTE

Safe does not mean deadlocked otherwise. An unsafe state simply means no guarantee of avoiding deadlock exists anymore. Avoidance refuses that gamble: it only grants a request if the resulting state is still provably safe.

The Resource-Allocation Graph Algorithm

- Extends Lecture 19's RAG with a new claim edge (dashed, like a request) meaning "this process might request this resource in the future"
- Every process must declare its claim edges before it runs
- When a process actually requests a resource, convert the claim edge into a real request edge
- Grant the request — converting it to an assignment edge — only if doing so would NOT create a cycle anywhere in the graph
- If granting would create a cycle, the request is deferred, even though the resource is currently free

Checking for a cycle before every single grant is exactly what makes this "avoidance" rather than "prevention."

The Banker's Algorithm: Four Data Structures

Available

A vector giving the number of currently free instances of each resource type

Max

Row i gives the maximum instances of each resource type P_i may ever request, declared in advance

Allocation

Row i gives the number of instances of each resource type currently allocated to P_i

Need

Row i gives additional instances P_i may still request: $Need[i] = Max[i] - Allocation[i]$

One identity must hold for every process, always: $Need = Max - Allocation$.

Three Resources (X=9, Y=6, Z=8), Five Processes

Allocation

Process	X	Y	Z
P0	1	2	2
P1	2	0	1
P2	2	1	1
P3	0	1	2
P4	1	1	0

Max

Process	X	Y	Z
P0	5	4	4
P1	3	2	2
P2	5	3	3
P3	2	2	3
P4	3	3	2

Need (Max – Allocation) and Available

Need = Max – Allocation

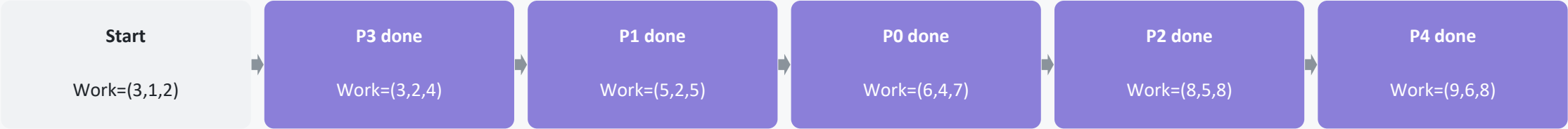
Process	X	Y	Z
P0	4	2	2
P1	1	2	1
P2	3	2	2
P3	2	1	1
P4	2	2	2

Available = Total – Σ Allocation

X	Y	Z
9 – 6 = 3	6 – 5 = 1	8 – 6 = 2

Column sums of Allocation: X = 1+2+2+0+1 = 6, Y = 2+0+1+1+1 = 5, Z = 2+1+1+2+0 = 6.

Finding a Safe Sequence by Hand



Step	Scan Finds	Why It Qualifies (Need ≤ Work)	Work After
1	P3	Need (2,1,1) ≤ (3,1,2)? Yes	(3,2,4)
2	P1	Need (1,2,1) ≤ (3,2,4)? Yes	(5,2,5)
3	P0	Need (4,2,2) ≤ (5,2,5)? Yes	(6,4,7)
4	P2	Need (3,2,2) ≤ (6,4,7)? Yes	(8,5,8)
5	P4	Need (2,2,2) ≤ (8,5,8)? Yes	(9,6,8)

Safe sequence: P3, P1, P0, P2, P4. The final Work vector (9,6,8) equals the system totals — a built-in arithmetic check.

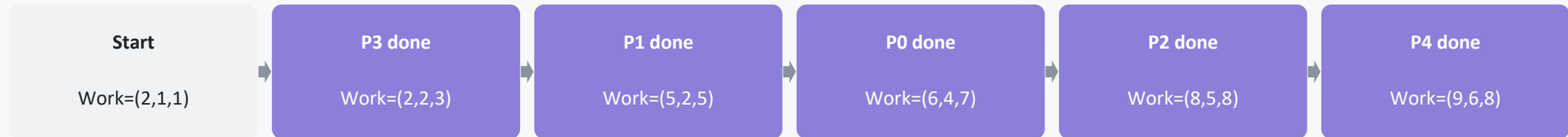
The Resource-Request Algorithm

- Check 1: is the request $\leq$ that process's remaining Need?
- Check 2: is the request $\leq$ current Available?
- If both pass, pretend-grant it — update Available, Allocation, and Need as if the grant happened
- Rerun the Safety Algorithm on that hypothetical state
- Only if the hypothetical state is still safe does the grant become permanent; otherwise roll back and the process waits

EXAMPLE A

P1 Requests (1, 0, 1) — Granted

- Check 1: $(1,0,1) \leq \text{Need}[P1]=(1,2,1)$? Yes
- Check 2: $(1,0,1) \leq \text{Available}=(3,1,2)$? Yes
- Pretend-grant: $\text{Available}'=(2,1,1)$, $\text{Allocation}'[P1]=(3,0,2)$, $\text{Need}'[P1]=(0,2,0)$



GRANTED

The hypothetical state is safe (sequence P3, P1, P0, P2, P4; Work returns to (9,6,8)), so P1's request is granted immediately and permanently.

EXAMPLE B

P2 Requests (2, 1, 0) — Denied

- Check 1: $(2,1,0) \leq \text{Need}[P2]=(3,2,2)$? Yes. Check 2: $(2,1,0) \leq \text{Available}=(3,1,2)$? Yes
- Pretend-grant: $\text{Available}'=(1,0,2)$, $\text{Allocation}'[P2]=(4,2,1)$, $\text{Need}'[P2]=(1,1,2)$

Process	Need	Need $\leq$ Work (1,0,2)?
P0	(4,2,2)	No — $4 > 1$
P1	(1,2,1)	No — $2 > 0$
P2	(1,1,2)	No — $1 > 0$
P3	(2,1,1)	No — $2 > 1$ and $1 > 0$
P4	(2,2,2)	No — $2 > 1$

DENIED

No process can proceed from $\text{Work}=(1,0,2)$, and Work can never grow without a process finishing first. The hypothetical state is unsafe, so the pretend-grant is rolled back and P2 must wait — even though enough resources were technically available right now.

Avoidance, Not Prevention

- 1 Avoidance differs from prevention in strategy, not goal: prevention forbids a condition permanently; avoidance dynamically refuses any allocation that would leave the system unsafe
- 2 A safe state has at least one safe sequence; an unsafe state is not automatically deadlocked, but it is no longer guaranteed deadlock-free
- 3 The RAG algorithm (claim edges, grant only if no cycle) handles single-instance resources; the Banker's Algorithm generalizes to multiple instances
- 4 The four structures relate by one identity, always: $\text{Need} = \text{Max} - \text{Allocation}$
- 5 The Safety Algorithm finds a safe sequence by repeatedly granting the first process whose Need fits Work; the Resource-Request Algorithm pretends to grant, then reruns it before committing

Lecture 21 covers what happens when the system cannot get this information in advance: detection and recovery.